
INFLUENCER OR OBSERVER: PREDICTING SOCIAL ROLES

TAGNE WAFFO Joel

Ecole Polytechnique

joel.tagne-waffo@polytechnique.edu

DEHAYEM KENFOUO Sylvain

Ecole Polytechnique

sylvain.dehayem-kenfouo@polytechnique.edu

SENG Hok

Ecole Polytechnique

hok.seng@polytechnique.edu

December 11, 2025

Introduction

In the era of digital marketing and information propagation, understanding user roles on social media platforms is critical for applications ranging from recommendation systems to viral content prediction. The distinction between an *Influencer* (a user capable of shaping opinions and driving engagement) and an *Observer* lies not only in the content they produce but also in their behavioral patterns and network characteristics.

In this project, we address the binary classification task of predicting whether a Twitter account acts as an Influencer or an Observer. The challenge involves a heterogeneous dataset containing numerical metadata, categorical attributes, and unstructured textual content. Our objective is to build a robust predictive model that effectively leverages this multimodal information.

Our approach relies on three main aspects. First, we performed extensive **Data Preprocessing and Feature Engineering**, uncovering a crucial insight: aggregating tweet-level statistics to the user level (grouping by account creation date) significantly reduces noise and improves user identification. Second, we integrated modern Natural Language Processing (NLP) techniques by utilizing Large Language Models such as **RoBERTa** and **Pythia-160M** to extract rich semantic embeddings from tweet text. Finally, we employed state-of-the-art Gradient Boosting Decision Trees (**LightGBM** and **XGBoost**), optimized via **Optuna** and combined through a bagging ensemble strategy.

This report details our methodology, from exploratory data analysis to the final ensemble architecture, demonstrating how combining handcrafted features with deep semantic representations yields superior classification performance.

1 Data Preprocessing & Feature extraction

1.1 Data analysis

The first step of the project was a thorough analysis of the data to understand insights and elements that could be relevant for the prediction. We tried to represent features, analyzed their relationships with the target. We remove feature with **zero variance** at all and also features with more than **90%** of data missing. During a later analysis we also found out that the feature `user.created_at` has a particularity and helped us increase our score on the public leaderbord. In fact our dataset has around **38k** unique users and this feature solely identify **30k** users. However all users with the same creation date had the same label, meaning that it is rare to find two differents users with the same account creation date. Also we looked at the user information for every tweet and we found out that all user with the same `user.created_at` shared almost the same User informations. However, we only have **30k** unique value of the feature instead of **38k** as mentioned in the description of the dataset.

1.2 Data Preprocessing

In this step, we evaluate the missing percentage of each feature among 192 features. We found out that 73 % of feature variables had more than 50% missing values . To solve and fill some missing values, we group our data according to the `user.created_at` which is considered as the **user id** and then used the median for numerical data and the most frequent element for others. Moreover, we securely remove the features whose variances are zero because they don't carry any information for the classification task.

1.3 Feature Engineering

Our feature engineering pipeline is implemented in the function `engineer_features` and focuses on constructing **user-level** and **tweet-level** statistics that turned out to be highly predictive in our final model.

User activity and popularity. From the raw Twitter metadata (`user.statuses_count`, `user.favourites_count`, `user.listed_count`, `user.followers_count`, `user.friends_count`, `user.created_at`), we derive several ratios and interaction terms that measure how visible and active a user is:

- **Account age** at tweet time, `account_age_days`,
- **Activity rates**, such as `statuses_per_day` and `followers_per_day`.

At training time, these features are further aggregated at user level (mean over all tweets from the same user), yielding variables such as `mean_user_listed`, `mean_user_statuses`, `mean_account_age_days` and `mean_total_activity`, which appear among the top predictors in our feature importance ranking for the **LightGBM Classifier**.

Text and profile statistics. Instead of modelling raw text directly in this tabular pipeline, we rely on simple but informative statistics:

- For the tweet text (`full_text`), we compute the character length and word count, then aggregate them per user (`mean_text_length`, `mean_text_word`, `std_text_length`).
- For the profile description (`user.description`), we compute analogous features (`user_description_length`, `user_description_word_count`, and their user-level means).

These length-based features, especially their **user-level averages and ratios**, are consistently ranked among the most important variables.

Temporal and behavioural patterns. We derive `tweet_hour` and `tweet_dayofweek` from the tweet timestamp. Aggregating these over users yields `mean_tweet_hour` and `std_tweet_hour`, which capture when a user is typically active and how concentrated their posting schedule is.

2 Training

2.1 Large language models

Beyond classical feature engineering, we experimented with several large language models to obtain richer text representations. We first evaluated four pretrained models on the task: BERT, RoBERTa, BERTweet (VinAI) and Pythia-160M (EleutherAI). All BERT-like models were fine-tuned in a standard sequence classification setting with two classes (Observer vs. Influencer), while Pythia was fine-tuned autoregressively. We also investigated LoRA fine-tuning for the Bert model but we did not pursue this direction further.

Encoder models (BERT / RoBERTa / BERTweet). For encoder-only architectures, we kept the tweet text (the `combined_text` field, which merges tweet content and user description) and formatted it as input to a binary classifier. Each model was fine-tuned with a cross-entropy loss on the influencer label. During inference, we used the representation of the special [CLS] token, which is the vector representing the encode text. These embeddings were concatenated with our tabular features. We considered three variants in our experiments: Roberta without fine-tuning (“0k” steps), Roberta fine-tuned, and other BERT/BERTweet baselines.

Decoder model (Pythia). Pythia-160M, a decoder-only transformer, was fine-tuned as a causal language model using a prompt-based formulation. For each training, example we built a prompt of the form:

```
What type of Twitter account posted this tweet?
{combined_text}
OPTIONS:
0: Observer
1: Influencer
ANSWER: {label}
```

The model was trained for 6000 steps to predict the correct answer token autoregressively. At inference time we dropped the label from the prompt and used the last hidden layer to construct a sentence embedding via attention-masked mean pooling over all non-padding tokens.

Integration with tabular features. For each tweet we therefore have (i) engineered tabular features (user activity statistics, temporal patterns, entity usage, etc.), and (ii) one or several LLM-based embeddings: BERTweet 0k, BERTweet 2k and Pythia 6k.

2.2 Feature Importance

To study the correlation between the predictors, we first extracted the feature importances from the LightGBM model using LLM-based text encoders (Pythia and RoBERTa), and from the XGBoost model with both encoders included. We observed that the features driving the classification differed significantly between the two models (see Appendix 3), which indicates that these weakly correlated models are well suited for bagging.

2.3 Bagging and Ensemble strategy

To improve robustness beyond a single classifier, we adopted a bagging-style ensemble strategy with two levels of diversification: (i) diversity in the **input representations** and (ii) diversity in the **training samples**.

Diversity through heterogeneous text encoders. Each model in Table 3 combines the same tabular features with a different Text encoder. As a consequence, even though many models use the same base learner (LightGBM or XGBoost), they rely on different subsets of information to make their predictions. This heterogeneity reduces the correlation of the errors between models, which is exactly what bagging tries to exploit.

Bagging over cross-validation folds. For each configuration in Table 3, we perform a 5-fold StratifiedGroupKFold at the user level. On each fold, the model is trained on $\frac{4}{5}$ of the users and evaluated on the remaining $\frac{1}{5}$. During training we also store the prediction probabilities on the test set for each fold. The final prediction of a given configuration on the test data was obtained by averaging the probabilities over the five folds. This is a form of *cross-validation bagging*: each base learner sees a slightly different bootstrap of users, which further decorrelates their errors.

2.4 HYPERPARAMETER TUNING WITH OPTUNA

We used Optuna to optimize the hyperparameters of our XGBoost and LightGBM models. At each iteration, the model is evaluated using 5-fold cross-validation, and the average validation score is returned to Optuna. This adaptive approach is significantly more efficient in terms of time complexity and computational cost compared to exhaustive methods such as grid search, while still enabling the discovery of high-performing hyperparameter configurations. We observed that the model Xgboost took more time than LightGBM because LightGBM uses histogram binning, leaf-wise tree growth, and memory-efficient algorithms, while XGBoost performed more exhaustive split evaluations.

2.5 Results

Table 3 summarizes the leaderboard accuracies obtained with the LLM-based setups. A first observation is that XGBoost has almost the same performance as LightGBM, but the latter is significantly faster than the former. So we conducted several of our analyses. with LightGBM. We can notice that introducing **user features** and using **Majority vote** to make the prediction uniform for the same user make our score increase significantly. Using features from two Large language model is better than only one, but the training time also increase. Training Bert with the LORA method was not significantly different from a trained RoBERTa with 2K steps. So we decided to choose only one of them.

Final ensemble. In the last step, we aggregate several of the strongest but complementary configurations by averaging their predicted probabilities. In particular, the *Final ensemble* in Table 3 combines:

- The best LightGBM models based on Roberta 2K and Pythia 6K embeddings with user features,
- The best XGBoost counterparts,

Individually, the strongest single models reach accuracies around 0.852–0.856, whereas the final ensemble attains 0.859. Although the gain is modest, it is consistent with the theoretical effect of bagging: by averaging several high-performing but only partially correlated models, we reduce variance and obtain a more stable and slightly more accurate predictor than any component model taken in isolation.

Overall, LLM embeddings bring complementary signal, but the best gains are obtained when they are combined with strong user-level features and simple ensembling rather than used in isolation.

Conclusion

In this study, we developed a comprehensive pipeline to classify Twitter users as Influencers or Observers, achieving a final accuracy of **0.859** on the leaderboard. Our experiments demonstrated that while advanced NLP models provide valuable semantic signals, they are most effective when combined with rigorous tabular feature engineering.

The key to our model’s performance was the discovery that user-level aggregation, specifically derived from unique identifiers like `user.created_at` which provided a far stronger signal than tweet-level features in isolation. Furthermore, our results highlight the efficacy of **ensemble learning**: by bagging models that utilize different text encoders (RoBERTa, BERTweet, Pythia) and different gradient boosting architectures (LightGBM, XGBoost), we were able to capitalize on uncorrelated errors and reduce variance.

Despite these successes, several avenues remain for future work. First, the computational cost of fine-tuning large language models remains high; investigating more efficient methods like **LoRA** or **distilled models** could improve scalability. Second, our current approach treats users as independent entities. Integrating **Graph Neural Networks (GNNs)** to model the explicit social graph (followers, friends, mentions) could unlock significantly deeper insights into the relational nature of influence. Finally, further optimization of categorical feature encoding could refine the model’s ability to distinguish user behaviors.

3 Appendix

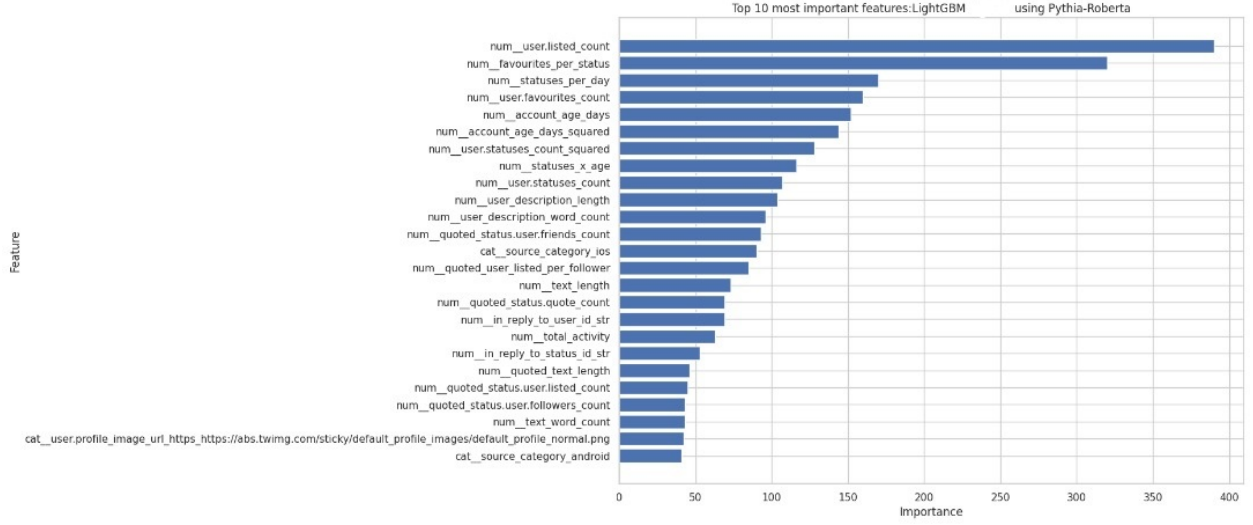


Figure 1: **Feature importance for LightGBM** With text embeddings, the model relies heavily on aggregated user behavioral statistics (e.g., mean_user_listed, favourites_per_status). This model captures *who* the user is based on their metadata history.

Model	Accuracy
XGB + Roberta 2k+ Pythia 6k	0.845
XGB + Roberta 0k+ Pythia 6k	0.845
XGB + Bert LORA Finetuning	0.843
LGBM + Roberta 2k+ Pythia 6k	0.846
LGBM + Roberta 3K+ Pythia 6k (with user features)	0.856
LGBM + Roberta 0k+ Pythia 6k	0.846
LGBM + Roberta 0k+ Pythia 6k (with user features)	0.852
LGBM + Roberta 2k	0.843
LGBM + Roberta 2k (with user features)	0.853
LGBM + Pythia 0k	0.840
LGBM + TFIDF (with user features)	0.850
LGBM + Bert LORA finetuning	0.843
LGBM + Bert Tweet 0k+ Pythia 6k (with user features)	0.853
Final ensemble (LGBM + XGBoost+ (Roberta 3K + Pythia 6K+ Bert Tweet 0k))	0.859

Table 1: **Comparative performance on the leaderboard.** The results demonstrate the critical importance of feature engineering: adding aggregated user features consistently yields significant gains (e.g., boosting the RoBERTa-based model to 0.856). The final ensemble achieves the highest accuracy by leveraging the diversity of multiple text encoders and gradient boosting implementations.

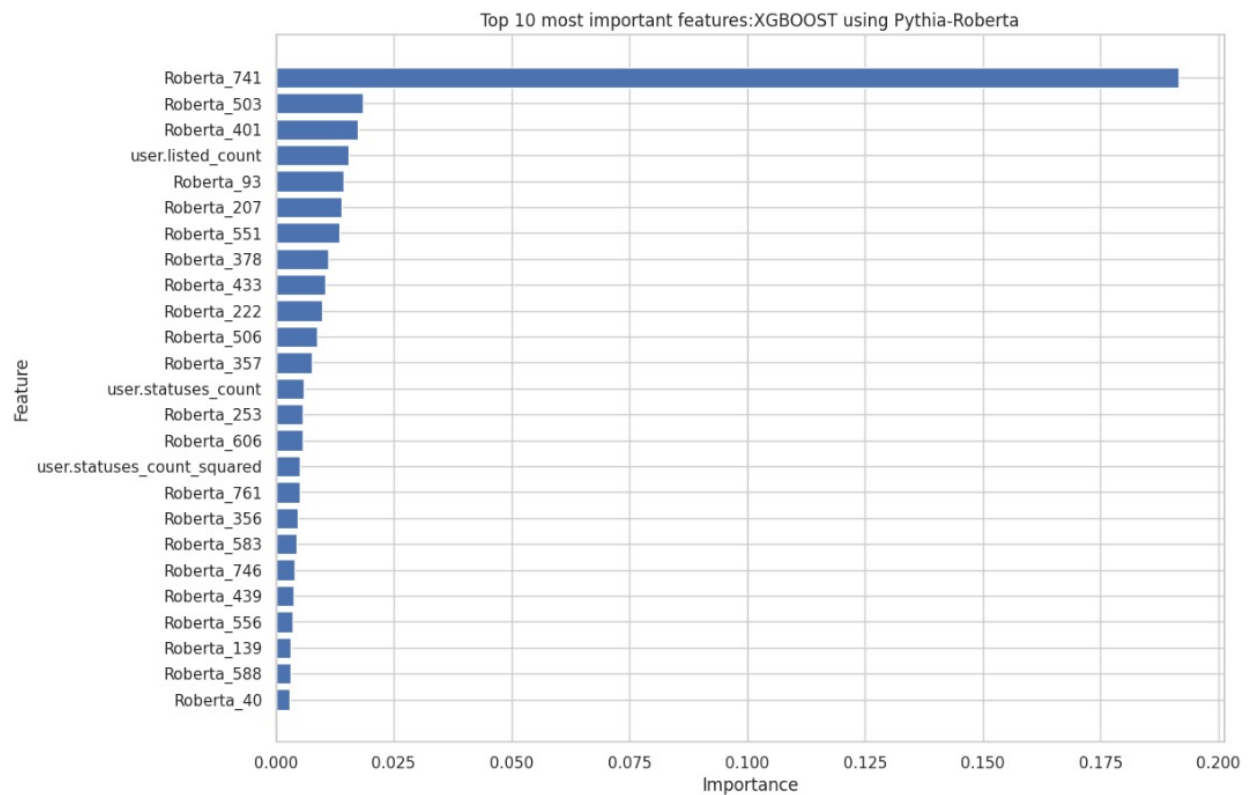


Figure 2: **Feature importance for XGBoost** When textual embeddings are introduced, the model shifts its focus entirely. The top predictors are specific dimensions of the RoBERTa embeddings (e.g., Roberta_741), indicating that this model captures *what* the user writes. The lack of overlap with Figure 1 confirms that the two models rely on uncorrelated signals, making them ideal candidates for bagging.

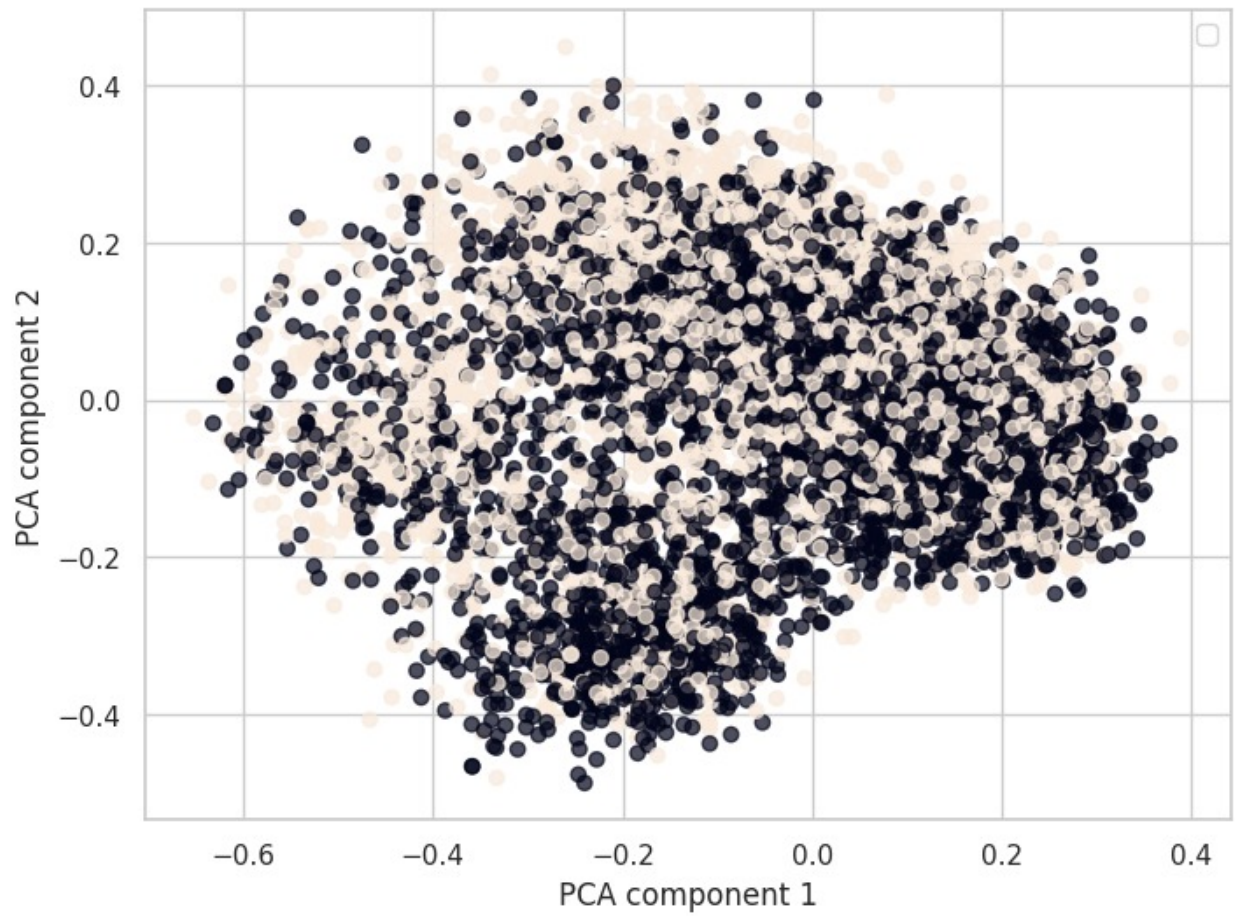


Figure 3: **PCA projection using TF-IDF embeddings.** The significant overlap between the two classes (represented by dark and light points) demonstrates the limitations of frequency-based text vectorization in linearly separating Influencers from Observers.

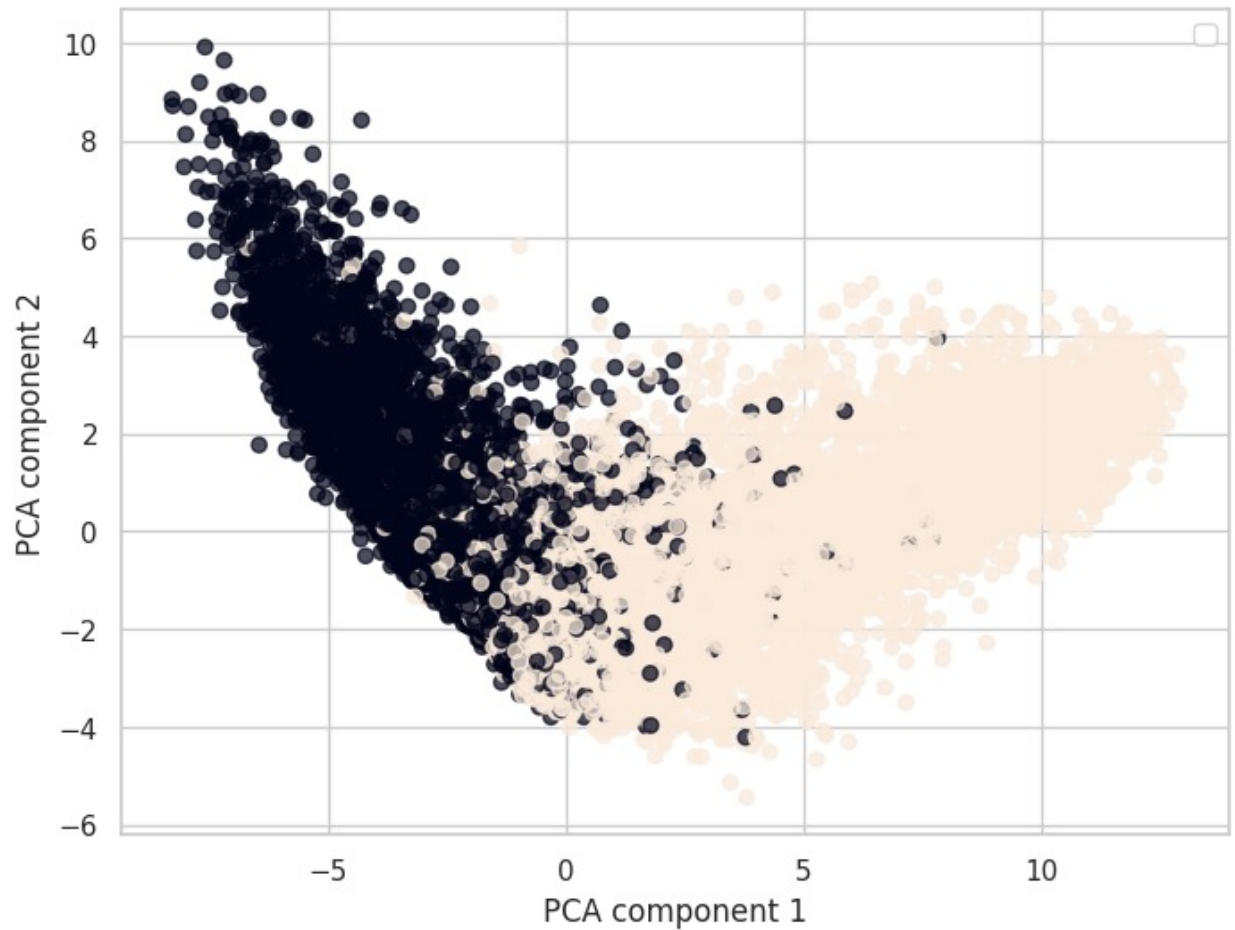


Figure 4: **PCA projection using fine-tuned BERT embeddings.** Unlike the TF-IDF approach, the deep semantic representations learned by the fine-tuned BERT model create a clear separation between the two classes, justifying the use of LLMs for this classification task.